Detección de piletas en imágenes satelitales con YOLOv5n

Transfer learning, métricas de detección y análisis crítico de resultados

Maestría en Inteligencia Artificial — Universidad de Palermo · Electiva: Visión por Computadora · Trabajo Práctico Final

---

1. Resumen ejecutivo

Se entrenó un detector de objetos de **una sola clase** (`pileta`) sobre imágenes satelitales, partiendo de pesos pre-entrenados en COCO y aplicando *fine-tuning* completo. El modelo elegido fue **YOLOv5n** (variante *nano*, 2.51 M parámetros), el más chico de la familia y el único que permite entrenar e inferir con holgura en el tier gratuito de Google Colab.

Resultado principal: mAP@0.5 = **0.7846** y F1 = **0.8094** en test, con 9.5 ms por imagen en GPU (104.8 FPS) y 5.23 MB de peso en disco.

Conclusión central: el modelo **clasifica bien pero localiza con imprecisión**. La brecha entre mAP@0.5 (0.78) y mAP@0.75 (0.40) es el hallazgo más informativo del experimento, y se explica principalmente por haber entrenado a 320 px de resolución.

El informe documenta además **tres defectos metodológicos** detectados al auditar el pipeline (§10), reportados de forma explícita en lugar de omitidos, por considerarse parte del valor académico del trabajo. Los tres están **corregidos en el código**, con verificaciones agregadas para que no se repitan en silencio; las métricas reportadas, en cambio, corresponden a la corrida original previa a esas correcciones, lo que las vuelve **conservadoras**.

2. El problema y los datos

2.1 Planteo

La tarea es **detección de objetos supervisada**: dada una imagen aérea, devolver las coordenadas de todas las piletas y una confianza para cada una. No es clasificación (no alcanza con decir "hay pileta") ni segmentación (no se requiere el contorno píxel a píxel), sino predicción de *bounding boxes*.

Es un problema de **clase única y fuertemente desbalanceado respecto del fondo**: una imagen satelital típica es casi toda techo, calle y vegetación, con una o dos piletas ocupando una fracción mínima del área. Esto condiciona el diseño: las métricas basadas en *accuracy* no sirven, y la elección del umbral de confianza pasa a ser una decisión central.

2.2 Dataset

Swimming Pool and Car Detection (Kaggle, `kbhartiya83`, licencia GPL 2).

Concepto	Detalle verificado en ejecución
Formato de anotación	XML Pascal VOC (un archivo por imagen)
Imágenes anotadas	3.748 · **16.172** *bounding boxes*
Clases crudas en los XML	`"1"` (~13.000 boxes) y `"2"` (~3.150 boxes)
Imágenes sin etiquetar	~10.200 (solo inferencia cualitativa)

2.3 El problema del mapeo de clases

Los XML identifican las clases con los literales `"1"` y `"2"`, **sin diccionario que indique cuál es cuál**.

Esto generó el error más costoso del trabajo: al asumir el mapeo sin verificarlo, el entrenamiento arrancó con **cero piletas en todos los splits**, produciendo un modelo que no detectaba nada. El diagnóstico fue confuso porque el pipeline no fallaba: entrenaba correctamente sobre un conjunto vacío de la clase de interés.

La solución fue introducir una **verificación visual obligatoria** antes de convertir los datos: se grafica una muestra con los boxes coloreados por clase cruda y se confirma a ojo cuál corresponde a piletas (rectángulos grandes, de color uniforme, junto a viviendas).



Verificado visualmente: `"2"` = **pileta**, `"1"` = auto. El código fija `CLASE_PILETA_XML = "2"` con un `assert` de existencia y descarta el resto.

Lección metodológica: en un pipeline automatizado, los errores silenciosos son más peligrosos que las excepciones. Una verificación visual de 30 segundos evitó horas de depuración sobre un modelo que funcionaba perfectamente mal.

2.4 Conversión a formato YOLO

YOLO no lee XML: requiere un `.txt` por imagen con una línea por objeto, `<clase> <cx> <cy> <w> <h>`, con las cuatro coordenadas **normalizadas entre 0 y 1** y `cx, cy` como centro del box (no la esquina, como en Pascal VOC).

La normalización usa el tamaño **real de cada imagen** leído con PIL, nunca un valor fijo: hardcodear el tamaño desplazaría todos los boxes de forma proporcional al error, produciendo un modelo que aprende a detectar piletas sistemáticamente corridas. Como control, el pipeline verifica que la cantidad de líneas escritas coincida exactamente con los boxes de clase pileta de los XML de cada split.

3. La familia YOLO: evolución y diferencias

3.1 El concepto fundacional

Antes de YOLO, los detectores dominantes (R-CNN, Fast/Faster R-CNN) eran de **dos etapas**: primero proponían regiones candidatas, después clasificaban cada una. Preciso, pero lento. YOLO —*You Only Look Once*— resolvió la detección como **un único problema de regresión**: una sola pasada de la red devuelve simultáneamente todas las clases y coordenadas. De ahí el salto de velocidad que habilitó la detección en tiempo real.

3.2 Evolución por generación

Versión	Año	Aporte principal	Cambio estructural clave
v1	2015	Detección en una sola pasada	Grilla 7×7, 2 boxes por celda, sin anclas
v2	2016	Anclas por k-means	*Anchor boxes*, batch norm, Darknet-19
v3	2018	Detección multiescala	3 escalas tipo FPN, 9 anclas, sigmoide multi-etiqueta
v4	2020	Optimización del entrenamiento	CSPDarknet53, mosaic, CIoU loss, Mish
v5	2020	Ingeniería y usabilidad	Reescritura en PyTorch, SPPF, variantes n/s/m/l/x
v6	2022	Orientado a producción	*Anchor-free*, cabeza desacoplada, RepVGG
v7	2022	Eficiencia arquitectónica	E-ELAN, cabezas auxiliares
v8	2023	Unificación multitarea	*Anchor-free*, módulo C2f, DFL, TAL

3.3 Los tres ejes que realmente diferencian las versiones

a) Anclas vs. *anchor-free. Un detector con anclas define rectángulos de referencia y predice el desplazamiento respecto de cada uno; introduce hiperparámetros sensibles al dominio (YOLOv5 los recalcula por k-means con auto-anchor*). Un detector* anchor-free*** predice directamente la distancia del centro a los cuatro bordes, eliminando esos hiperparámetros a costa de una asignación de objetivos más compleja.

b) Cabeza acoplada vs. desacoplada. En versiones tempranas una misma rama predecía clase y coordenadas. Desde v6/v8 se usan **ramas separadas**: clasificar y localizar tienen gradientes de naturaleza distinta, y separarlas mejora ambas.

c) Pérdida de localización. Error cuadrático sobre coordenadas (v1-v3) → **IoU/CIoU loss** (v4-v5), que optimiza directamente la métrica de evaluación → **DFL** (*Distribution Focal Loss*, v8), que modela cada borde como una **distribución de probabilidad discreta** en vez de un valor puntual. DFL mejora la precisión fina de localización.

3.4 Precisión importante sobre el modelo usado

El pipeline carga `yolov5nu.pt`, no `yolov5n.pt`. La diferencia no es cosmética: la `u` indica que es el *backbone* de YOLOv5 **actualizado con la cabeza anchor-free desacoplada de YOLOv8**, incluyendo pérdida DFL.

Evidencia empírica en nuestros propios datos: el `results.csv` del entrenamiento contiene la columna `train/dfl_loss`. El YOLOv5 original *no tiene* esa pérdida —es exclusiva de la cabeza v8—, por lo que su presencia confirma la arquitectura híbrida.

Esto corrige una afirmación incorrecta de la documentación previa del proyecto, que describía el modelo como "detector de una etapa **con anclas**". El modelo efectivamente entrenado es **anchor-free**. La distinción importa porque explica por qué no hubo que ajustar anclas para objetos pequeños, y porque la cabeza DFL es responsable de la calidad de localización analizada en §8.

3.5 Por qué YOLOv5n

Con ~1.300 imágenes de entrenamiento, un modelo grande sobreajustaría: la elección se fundamenta en la **relación entre capacidad del modelo y volumen de datos**. Aumentar la capacidad con pocos datos no mejora la generalización, la empeora. A esto se suman restricciones prácticas: 2.51 M parámetros, 10–15 min de entrenamiento en una T4 gratuita e inferencia viable en CPU (~27 FPS).

4. Arquitectura y configuración

4.1 Estructura del modelo

Todo detector YOLO moderno se organiza en tres bloques: el **backbone** extrae características jerárquicas (bordes → texturas → formas → objetos); el **neck** fusiona información entre escalas (PANet + SPPF); la **head** produce las predicciones en **tres escalas** simultáneas —mapas de 40×40, 20×20 y 10×10 para una entrada de 320 px—.

La detección multiescala es lo que permite encontrar piletas pequeñas: la escala de 40×40 aporta la resolución espacial necesaria, mientras la de 10×10 aporta contexto semántico amplio.

4.2 Hiperparámetros utilizados

Parámetro	Valor	Motivo
Pesos iniciales	`yolov5nu.pt` (COCO)	Transfer learning
`imgsz` / `batch`	320 / 8	Límite de memoria y tiempo en T4
`epochs` / `patience`	25 (máx.) / 6	Presupuesto de cómputo + early stopping
`seed` / `nc`	42 / 1	Reproducibilidad · una sola clase

Épocas ejecutadas: 25 de 25. El early stopping **no se activó**, dato relevante: el modelo seguía mejorando cuando se agotó el presupuesto. La mejor época fue la **20**.

Sobre `nc=1`: fijar una sola clase reduce la cabeza al mínimo y, más importante, **elimina la posibilidad de confusión entre clases**. Los autos no se descartan por ser irrelevantes, sino porque incluirlos obligaría al modelo a resolver un problema adicional sin beneficio para el objetivo.

5. Transfer learning: capas congeladas y no congeladas

5.1 El concepto

Una red entrenada en un conjunto masivo (COCO: 330.000 imágenes, 80 clases) aprende una **jerarquía de representaciones reutilizable**: las **capas iniciales** detectan bordes, gradientes y texturas —conocimiento **genérico**, un borde es un borde en cualquier dominio—; las **intermedias**, patrones compuestos y partes de objetos; las **finales**, combinaciones específicas de la tarea original.

Congelar una capa significa marcar sus pesos como **no entrenables**: participan del *forward pass* pero no reciben actualización en el *backward pass*.

```
for parametro in capa.parameters():
    parametro.requires_grad = False
```

La intuición: si las primeras capas ya saben detectar bordes, reentrenarlas con 1.300 imágenes solo puede degradarlas.

5.2 Las tres estrategias

Estrategia	Qué se entrena	Cuándo conviene
Feature extraction	Solo la cabeza	Pocos datos (<500 img), dominio similar al original
Fine-tuning parcial	Cabeza + capas superiores	Caso intermedio, el más común
Fine-tuning completo	Toda la red	Suficientes datos o dominio muy distinto

5.3 Cómo se aplica en Ultralytics

El framework expone el parámetro `freeze`, que congela los primeros *N* módulos:

El framework congela los parámetros cuyo nombre coincide con `model.{i}.` para *i* en `range(freeze)`, más siempre la rama `.dfl`:

```
freeze_layer_names = [f"model.{x}." for x in freeze_list] + [".dfl"]
```

Los valores de referencia **se verificaron cargando el modelo**, no se asumieron. `yolov5nu.pt` tiene **25 módulos (0–24)** y `Detect` es el **24**:

`freeze`	Parámetros entrenables	Qué queda libre
0	2.636 M (**100 %**)	Toda la red: fine-tuning completo
10	1.589 M (**60.3 %**)	Neck + head
24	0.878 M (**33.3 %**)	Solo la cabeza `Detect`
25	0 M	Ultralytics aborta con `RuntimeError`

Dos detalles no obvios que surgieron al verificarlo:

1. `freeze=22` **no equivale a "solo la cabeza"**, aunque sea el valor que suele citarse. Deja los módulos 23 **y** 24 entrenables (45 %). Para *feature extraction* real sobre esta arquitectura corresponde `freeze=24`.
2. **El conteo de parámetros no puede hacerse sobre el modelo recién cargado.** `yolov5nu.pt` viene con `nc=80` y Ultralytics reconstruye la cabeza para `nc=1` al entrenar; contar antes sobreestima en 19.200 parámetros. La medición correcta usa un callback que se ejecuta cuando el *freeze* ya se aplicó y el modelo todavía no se fusionó:

```
def _on_train_start(trainer):
    params = list(trainer.model.parameters())
    cap["entrenables"] = sum(p.numel() for p in params if p.requires_grad)
    cap["total"] = sum(p.numel() for p in params)

modelo.add_callback("on_train_start", _on_train_start)
```

La verificación del porcentaje entrenable conviene hacerla siempre: si no cambia al modificar `freeze`, la configuración no se está aplicando.

5.4 Ventajas y desventajas

Ventajas de congelar

1. **Menor consumo de memoria.** No se almacenan gradientes ni estados del optimizador de las capas congeladas; permite subir `batch size` o resolución con la misma GPU.

2. **Entrenamiento más rápido.** El *backward pass* es la parte costosa; recortarlo acelera cada época.
3. **Protección contra sobreajuste.** Menos parámetros libres frente a pocos datos reduce la varianza.
4. **Evita *catastrophic forgetting*.** Con *learning rates* altos y pocos datos, el fine-tuning completo puede destruir representaciones genéricas útiles.
5. **Mayor estabilidad.** Menos grados de libertad implica menor dispersión entre corridas.

Desventajas de congelar

1. **Techo de rendimiento.** Si el dominio nuevo difiere del original, las características congeladas pueden ser inadecuadas y el modelo no puede corregirlas.
2. **Problema directo en este caso: COCO es de nivel de suelo, el dataset es aéreo.** Vista cenital, escala uniforme, iluminación homogénea, sin perspectiva. Las texturas de COCO no transfieren tan bien como lo harían entre dos datasets de fotografía convencional.
3. **Hiperparámetro adicional.** Determinar cuántas capas congelar con rigor requiere experimentación.
4. **Interacción con BatchNorm.** Las capas de normalización mantienen estadísticas móviles que pueden seguir actualizándose aunque sus pesos estén congelados, generando comportamiento inconsistente entre entrenamiento e inferencia.

5.5 Decisión adoptada

Se entrenó sin congelar ninguna capa (fine-tuning completo). Fundamento: la **distancia de dominio** entre COCO y las imágenes satelitales es grande —las piletas vistas desde arriba son rectángulos de color saturado y uniforme, sin análogo directo en COCO—. Permitir que el *backbone* se adapte es razonable cuando las características de bajo nivel relevantes difieren de las del dominio original. El resultado (mAP@0.5 = 0.78 en 25 épocas) es consistente con que la estrategia fue adecuada.

5.6 Estado del experimento comparativo

> **Declaración de alcance.** Las corridas de entrenamiento con esta corrida base **no incluyeron una comparación con capas congeladas**, por lo que las métricas de §7 corresponden únicamente a `freeze=0`. **Este informe no presenta mAP medido para `freeze=10` ni `freeze=24`.** Se evita deliberadamente presentar cifras de literatura como si fueran resultados propios.
> Lo que sí está **medido y verificado** es la mecánica: el recuento de parámetros entrenables de cada estrategia (tabla de §5.3) y la cantidad de tensores congelados en cada caso (1 / 100 / 172), obtenidos cargando el modelo real y ejecutando el *trainer*.
> El experimento quedó **implementado y probado** como Etapa 4B del pipeline (`CORRER_FREEZE`), con las tres configuraciones a igual número de épocas y semilla, registrando mAP, tiempo por época, memoria pico y porcentaje entrenable en `resultados/freeze_comparativa.csv`. Falta ejecutarlo sobre el dataset completo en GPU; al disponer de esos números, esta sección pasa de teoría verificada a resultado comparativo (§11).

6. Métricas de detección: definición e interpretación

Cada métrica responde una pregunta distinta. Usarlas sin precisar la pregunta es la fuente más común de conclusiones erróneas.

IoU — *Intersection over Union*. Base de todo lo demás: `IoU = área de intersección / área de unión`, entre 0 (sin contacto) y 1 (coincidencia exacta). Un IoU de 0.5 implica solapamiento considerable con desajuste visible; 0.75 corresponde a un ajuste geométricamente fino. **Es el árbitro que convierte una predicción en acierto o error**: cambiando el umbral, la misma predicción pasa de TP a FP sin que el modelo haya cambiado.

TP, FP, FN y la ausencia de TN. Un **TP** es una pileta detectada con IoU ≥ umbral; un **FP**, un aviso sin pileta real o con box mal ubicado; un **FN**, una pileta real no detectada. El **TN no está definido**: en clasificación sería "predijo negativo y era negativo", pero un detector **no emite predicciones negativas** —cada región de fondo correctamente ignorada sería un TN, y hay infinitas regiones posibles por imagen—. **Consecuencia directa:** no se pueden usar *accuracy* ni especificidad, porque ambas requieren TN. Por eso la evaluación de detectores se construye sobre precisión, recall y mAP.

Precisión = `TP / (TP + FP)`. De todo lo que el modelo señaló, qué fracción era correcta: *¿puedo confiar en sus avisos?* Precisión baja implica ruido que un operador humano debe descartar.

Recall = `TP / (TP + FN)`. De todo lo que existía, qué fracción encontró: *¿se le escapan cosas?* Recall bajo implica omisiones, que en algunos contextos son el error más costoso.

F1-score = `2 × (P × R) / (P + R)`. Media **armónica**, elegida deliberadamente porque penaliza el desbalance: con P=1.0 y R=0.1, la media aritmética da 0.55 —engañosamente aceptable—, mientras F1 da **0.18**, reflejando que el modelo es inútil en la práctica.

AP — *Average Precision*. Precisión y recall dependen del umbral elegido; AP resume el rendimiento **en todos los umbrales**, como área bajo la curva Precisión-Recall. Interpretación: **qué tan bien el modelo ordena sus predicciones**. Un AP alto significa que las detecciones correctas reciben, en general, puntajes más altos que las incorrectas. Si una predicción falsa tuviera la confianza más alta, el AP caería sin que el modelo cambiara ninguna detección.

Cálculo del AP paso a paso. Para que 0.7846 no quede como un número opaco, se reproduce el procedimiento con un ejemplo mínimo: 5 predicciones sobre un conjunto con 4 piletas reales. Las predicciones se **ordenan por confianza descendente** y se evalúa cada una con IoU ≥ 0.5:

#	Confianza	¿IoU ≥ 0.5?	TP ac.	FP ac.	Precisión	Recall
1	0.95	Sí	1	0	1/1 = **1.000**	1/4 = **0.25**
2	0.88	Sí	2	0	2/2 = **1.000**	2/4 = **0.50**
3	0.71	No	2	1	2/3 = **0.667**	2/4 = **0.50**
4	0.54	Sí	3	1	3/4 = **0.750**	3/4 = **0.75**
5	0.31	No	3	2	3/5 = **0.600**	3/4 = **0.75**

Dos observaciones sobre la mecánica. La **precisión fluctúa** al avanzar por la lista (baja con cada FP, sube con cada TP), mientras el **recall solo puede crecer o mantenerse**: nunca decrece, porque los TP acumulados no se pierden. Y el denominador del recall (4) es la cantidad de piletas **reales**, no de predicciones: una pileta nunca detectada no aparece en la tabla pero sí penaliza el recall final, cuyo máximo alcanzado es 0.75 y no 1.0.

El AP es el **área bajo esta curva**. Con interpolación a todos los puntos de recall, se toma para cada nivel la precisión máxima alcanzada a ese recall o superior:

```
AP = (0.25 - 0.00) × 1.000      ← precisión máx. con recall ≥ 0.25
   + (0.50 - 0.25) × 1.000      ← precisión máx. con recall ≥ 0.50
   + (0.75 - 0.50) × 0.750      ← precisión máx. con recall ≥ 0.75
   = 0.250 + 0.250 + 0.1875 = 0.6875
```

mAP y sus variantes. mAP promedia el AP sobre las clases; con una sola clase, **mAP = AP** —por eso `mAP_50` y `AP_pileta_50` tienen idéntico valor (0.7846): no es un error de cálculo sino una identidad algebraica—. Las variantes se diferencian por el IoU exigido:

Métrica	Umbral IoU	Qué evalúa
mAP@0.5	0.5	Detección: ¿encontró el objeto?
mAP@0.75	0.75	Localización precisa: ¿el box ajusta bien?
mAP@[0.5:0.95]	Promedio de 10 umbrales	Métrica estándar COCO, la más exigente

La comparación entre mAP@0.5 y mAP@0.75 es diagnóstica: si ambas son similares, el modelo localiza con precisión; si mAP@0.75 cae mucho, encuentra los objetos pero no los delimita bien. Es exactamente lo que ocurre acá.

mAR@100. Recall promedio permitiendo hasta 100 detecciones por imagen: el **techo de recall alcanzable** si se ignorara la precisión. Valor obtenido: 0.5173.

NMS — supresión de no-máximos. Un detector produce miles de predicciones solapadas, porque varias celdas y escalas detectan el mismo objeto. NMS ordena por confianza, acepta la mayor y elimina toda predicción cuyo IoU con ella supere un umbral. Importa no confundir los tres parámetros en juego: `conf=0.25` es la confianza mínima para emitir una detección; `iou=0.5` de NMS es el solapamiento a partir del cual se descarta un duplicado (decisión de **post-procesado** del modelo); el IoU de evaluación 0.5 es el criterio de **medición** externo al modelo.

mAP vs. P/R/F1: dos preguntas distintas. El mAP mide la calidad del *ranking* y se calcula sin umbral práctico (`conf=0.001`, sin filtrar casi nada, para construir la curva PR completa); P/R/F1 miden el rendimiento de la **decisión** con un umbral operativo fijo. El primero sirve para comparar modelos; el segundo, para operarlos.

Selección del umbral: por qué en validación y no en test. Se barren umbrales de 0.25 a 0.90 (paso 0.05) **sobre validación**, eligiendo el de mayor F1. Elegirlo mirando el test convertiría al test en parte del ajuste: la métrica reportada dejaría de estimar el rendimiento sobre datos no vistos y sería optimista por construcción. Es una fuga de información sutil y difícil de detectar en reportes ajenos. Por eficiencia, se predice sobre validación **una sola vez** con `conf=0.01` y luego se filtra por score en cada umbral; reinferir en cada uno multiplicaría el cómputo por 14 sin aportar nada.

Umbral óptimo obtenido: 0.25 — el **extremo inferior del rango barrido**, lo que constituye una señal de advertencia analizada en §10.3.

7. Resultados medidos

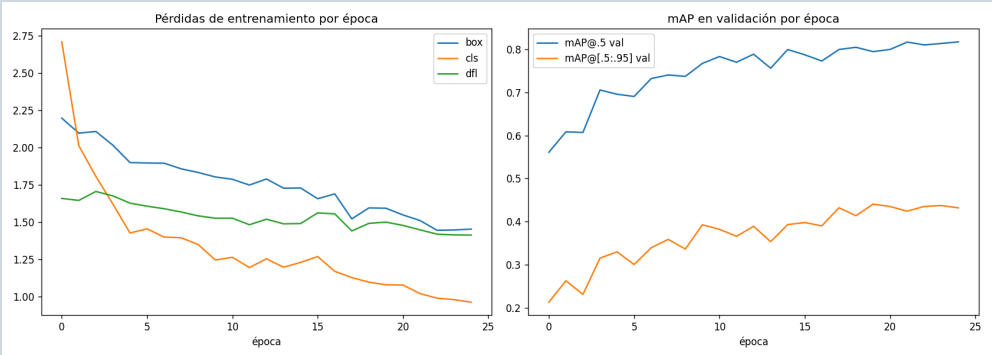
Todos los valores provienen de `metricas.csv`, generado por código durante la ejecución. Ninguno fue transcrito manualmente.

7.1 Cada dato y su significado

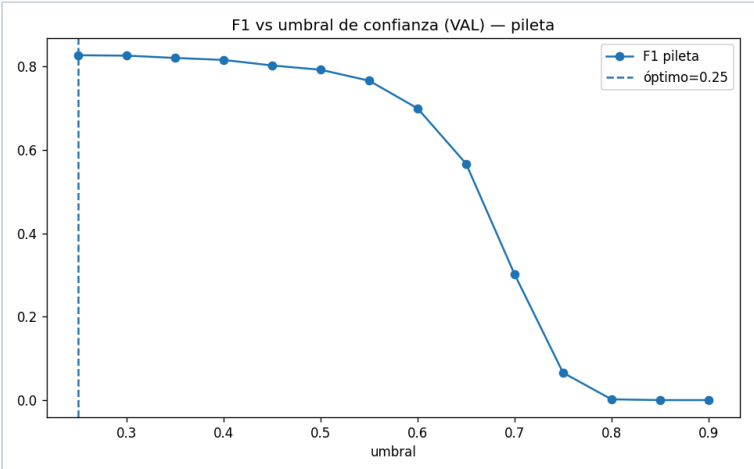
Las 16 métricas registradas sobre test (1.125 imágenes, umbral 0.25, IoU 0.5):

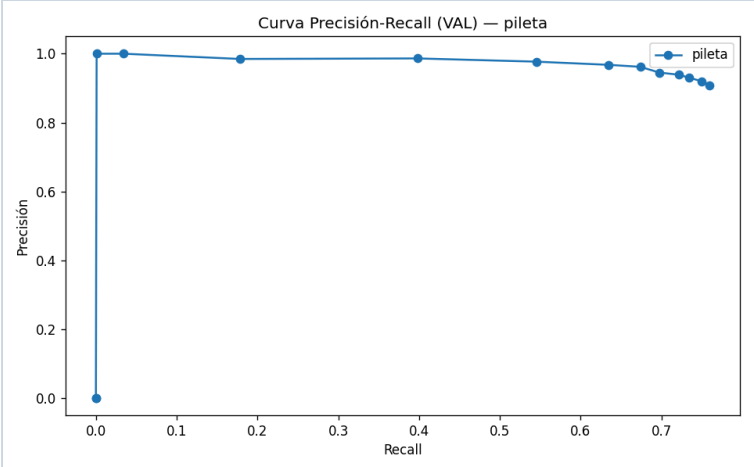
Métrica	Valor	Qué es	Qué significa este valor concreto
`mAP_50_95`	0.4203	AP promediado sobre IoU 0.50–0.95	Métrica COCO estándar. Razonable para un nano a 320 px
`mAP_50`	0.7846	AP con IoU ≥ 0.50	Encuentra ~78 % de las piletas con solapamiento aceptable
`mAP_75`	0.4003	AP con IoU ≥ 0.75	Cae 49 % vs. mAP@0.5: los boxes no ajustan con precisión
`mAR_100`	0.5173	Recall promedio, máx. 100 det./img	Techo de recall si no importara la precisión
`AP_pileta_50`	0.7846	AP de la clase con IoU 0.5	Idéntico a `mAP_50`: hay una sola clase
`P_pileta`	0.8912	Precisión	De 100 avisos, ~89 son reales (21 FP sobre 193)
`R_pileta`	0.7414	Recall	Encuentra 172 de 232 piletas: pierde 60
`F1_pileta`	0.8094	Media armónica de P y R	Balance sólido, sin desbalance severo
`IoU_prom_pileta`	0.7868	IoU medio de los TP	Cuando acierta ajusta bien; falla en lo que no detecta
`umbral_optimo`	0.25	Confianza que maximiza F1 en val	**Borde del rango barrido**: el óptimo real sería menor
`ms_por_img_GPU`	9.54	Latencia en T4	Con warmup y sincronización CUDA
`FPS_GPU`	104.80	Imágenes por segundo en GPU	Holgadamente apto para video en tiempo real
`ms_por_img_CPU`	49.17	Latencia en CPU	Sin warmup; el valor estabilizado es ~37 ms (§9.3)
`parametros_M`	2.51	Millones de parámetros	El menor de la familia
`peso_MB`	5.23	Tamaño del `best.pt`	Medido del archivo real
`GFLOPs`	0.897	Operaciones por inferencia	~3 órdenes de magnitud menos que un modelo grande

7.2 Curvas de entrenamiento y barrido de umbrales



Las **pérdidas** (`box_loss`, `cls_loss`, `dfl_loss`) descienden de forma sostenida, sin oscilaciones ni divergencia: el *learning rate* fue adecuado. El **mAP de validación** crece y se estabiliza, con máximo en la época 20. **No hay indicios de sobreajuste** —el patrón característico sería mAP de validación en descenso mientras la pérdida de entrenamiento sigue bajando—.





La curva F1 tiene su máximo en el borde izquierdo del rango (0.25) y decrece al subir el umbral: el modelo se vuelve más conservador, gana precisión y pierde más recall del que compensa. La curva PR muestra el *trade-off* subyacente: cada punto es una política operativa distinta, y elegir entre ellas no es una decisión estadística sino **de negocio** —depende del costo relativo de una falsa alarma frente a una omisión—.

8. Matriz de confusión y análisis de errores

8.1 Reconstrucción de la matriz

Las métricas de precisión y recall permiten recuperar los conteos absolutos. Resolviendo `P = TP/(TP+FP) = 0.8912` y `R = TP/(TP+FN) = 0.7414`, se obtiene la única solución entera consistente:

	Predicho: pileta	Predicho: nada
Real: pileta	**TP = 172**	**FN = 60**
Real: nada	**FP = 21**	TN — no definido

Totales: 232 piletas reales en test · 193 detecciones emitidas · 1.125 imágenes. Verificación: `172/193 = 0.8912` ✓ · `172/232 = 0.7414` ✓

8.2 Interpretación del perfil de error

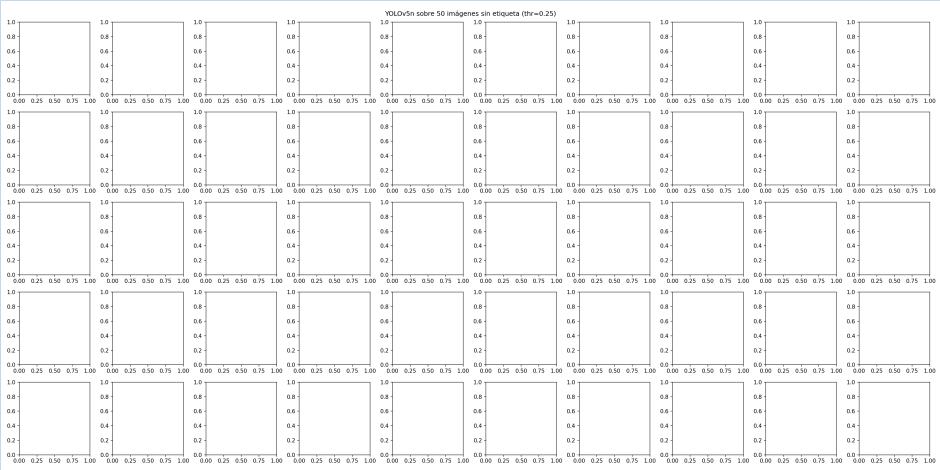
El dato más relevante es la **asimetría**: 60 falsos negativos frente a 21 falsos positivos. El modelo **falla casi tres veces más por omisión que por exceso**: es un detector **conservador**, confiable cuando avisa, pero que calla demasiado.

Dos matices importantes. Primero, **los FP no son necesariamente "inventos"**: un falso positivo puede ser una pileta correctamente identificada pero con IoU inferior a 0.5. El modelo *vio* la pileta y falló al delimitarla, pero la métrica lo cuenta igual que si hubiera señalado un techo vacío; dado el bajo mAP@0.75, parte de los 21 FP es probablemente de este tipo. Segundo, **los FN probablemente concentran piletas pequeñas**: a 320 px, una pileta chica ocupa muy pocos píxeles tras el redimensionado, quedando bajo el umbral de detectabilidad.

8.3 Inspección visual y generalización



Convención: rojo fino = *ground truth*, azul = predicción con su score. La inspección visual cumple una función que las métricas no pueden: un mAP de 0.78 es compatible con errores estructurales —boxes desplazados sistemáticamente, o una familia entera de piletas ignorada— que un único número promedio oculta.



Sobre las ~10.200 imágenes sin etiquetas **no existen métricas posibles** por ausencia de *ground truth*; el valor es cualitativo: verificar que el modelo no se degrada al cambiar de zona geográfica. Señal de alerta a observar: acumulación de falsos positivos sobre superficies de agua o techos azules indicaría que el modelo aprendió "rectángulo azul" en lugar de "pileta".

9. Costo computacional y exportación a ONNX

9.1 Complejidad y velocidad

El modelo tiene **2.51 M parámetros**, pesa **5.23 MB** y requiere **0.897 GFLOPs** por inferencia —aproximadamente **tres órdenes de magnitud menos** que un detector grande tipo YOLOv8x—. Velocidad medida: **9.54 ms/imagen en GPU T4 (104.8 FPS)** y **49.17 ms en CPU**.

Protocolo: 5 imágenes de *warmup* y promedio sobre 30 inferencias con `torch.cuda.synchronize()`. La sincronización es imprescindible: sin ella se mide el tiempo de encolado de las operaciones CUDA, no el de ejecución, y los resultados aparecen absurdamente optimistas.

9.2 Exportación a ONNX

ONNX (*Open Neural Network Exchange*) es un formato intermedio que desacopla el modelo de PyTorch, permitiendo ejecutarlo en runtimes diversos (ONNX Runtime, OpenVINO, TensorRT) sin dependencias del framework de entrenamiento.

Indicador	`.pt`	`.onnx`	Diferencia
Peso	5.23 MB	**10.14 MB**	+94 %
FPS en CPU	26.72	**33.85**	+27 %

ONNX es 27 % más rápido en CPU, consistente con lo esperado: ONNX Runtime aplica optimizaciones de grafo (fusión de operadores, eliminación de nodos redundantes) que PyTorch en modo *eager* no realiza. **Pesa casi el doble** porque el `.pt` almacena pesos comprimidos y reconstruye la arquitectura desde código, mientras el `.onnx` serializa el grafo completo de forma explícita.

La comparación de mAP del CSV no es válida (`map50_pt = 0.7846` vs. `map50_onnx = 0.8710`, delta +0.0864). Se analiza en §10.2.

9.3 Nota sobre la medición de FPS en CPU

Los dos archivos reportan velocidades de CPU distintas para el mismo modelo: 20.3 FPS en `metricas.csv` y 26.7 en `trade_off_onnx.csv`. No es contradictorio: la primera medición se realiza **sin warmup**, incluyendo el costo de inicialización del backend de CPU. **El valor correcto es ~27 FPS.** Se documenta porque reportar la primera cifra sin advertencia subestima el rendimiento real en un 24 %.

10. Limitaciones y hallazgos de la auditoría

Tres defectos metodológicos detectados al revisar el pipeline. Se incluyen por transparencia: un informe que oculta las debilidades de su propio diseño tiene menos valor técnico que uno que las identifica.

> **Estado.** Los tres defectos **ya están corregidos en el código** (`visionxcomputadora.py` y `VisionXComputadora.ipynb`). Las métricas de §7, en cambio, **provienen de la corrida original, anterior a las correcciones**: reentrenar requiere GPU y no se volvió a ejecutar el pipeline. Por eso cada apartado distingue el **defecto medido** de la **corrección aplicada**, y las cifras reportadas deben leerse como el resultado del pipeline defectuoso.

10.1 El split real es 35/35/30, no 70/15/15

La documentación declara una partición 70/15/15. Los conteos efectivos la contradicen: train = 1.311 (**35 %**), validación = 1.312 (**35 %**), test = 1.125 (**30 %**).

Causa raíz (`visionxcomputadora.py`, versión original, Etapa 3):

```
resto, df_test = split_seguro(df_img)                # 70 / 30
df_train, df_val = train_test_split(resto, test_size=0.50)   # divide el 70 % a la mitad
```

El segundo `train_test_split` se aplica sobre `resto` (el 70 %) y lo parte por mitades, dando 35 % y 35 %. El error está en **cuál de los dos valores devueltos se vuelve a partir**: para obtener 70/15/15 hay que dividir el **30 %**, no el 70 %.

Corrección aplicada:

```
df_train, temp = split_seguro(df_img)                          # 70 / 30
df_val, df_test = train_test_split(temp, test_size=0.50, ...)  # 15 / 15

assert abs(len(df_train) / n_tot - 0.70) < 0.02
assert abs(len(df_val) / n_tot - 0.15) < 0.02
assert abs(len(df_test) / n_tot - 0.15) < 0.02
assert not (s_tr & s_va) and not (s_tr & s_te) and not (s_va & s_te)
```

Se agregaron los *asserts* porque el bug original era silencioso: sin una verificación explícita de las proporciones, el pipeline completo corre sin un solo error con el split equivocado. Los porcentajes ahora también se imprimen.

Impacto: el modelo se entrenó con **1.311 imágenes en lugar de ~2.600**, la mitad de los datos disponibles, y el conjunto de validación quedó innecesariamente grande. **Implicación positiva:** las métricas reportadas son **conservadoras** —se alcanzó mAP@0.5 = 0.78 con la mitad de los datos previstos—. Reentrenar con el split corregido debería mejorar los resultados, especialmente el recall, a costa de duplicar el tiempo de entrenamiento (de 10–15 min a 20–30 min en T4).

10.2 El delta de mAP del ONNX no es comparable

El CSV reporta +0.0864 de mejora al pasar a ONNX. **El valor carece de significado**, porque las dos cifras se midieron sobre conjuntos distintos: el `.pt` sobre test completo (1.125 imágenes) y el `.onnx` sobre un subconjunto de validación (100 imágenes).

Además el resultado es implausible por diseño: **la exportación a ONNX sin cuantización preserva los pesos en FP32**. Es la misma red en otro formato, por lo que el delta esperado es ≈ 0 (ruido numérico). Un +8.6 puntos solo puede provenir de que esas 100 imágenes resultaron más fáciles que el test completo.

Corrección aplicada: ambos modelos se evalúan sobre el mismo subconjunto, y el CSV guarda por separado el valor de referencia sobre test completo para que no se los vuelva a confundir.

```
sub_cmp = imgs_test[:100]
pv_pt, tv_pt = coleccionar_pares(sub_cmp, conf=0.001)
pv, tv = coleccionar_pares(sub_cmp, conf=0.001, modelo_ref=modelo_onnx)
# CSV: map50_pt_sub, map50_onnx_sub, delta_map50,
#      n_imgs_comparacion, map50_pt_test_completo
```

El principio general: una comparación A vs. B solo es válida si **lo único que cambia entre las dos mediciones es A y B**. Acá cambiaban simultáneamente el formato del modelo, el conjunto de datos y su tamaño.

10.3 El barrido de umbrales no cubre el óptimo real

El umbral óptimo resultó **0.25**, el **límite inferior del rango explorado** (0.25–0.90). Cuando una optimización devuelve un valor en el borde de su dominio de búsqueda, lo más probable es que el óptimo verdadero esté **fuera** de ese dominio: la curva F1 es decreciente en todo el rango, sugiriendo un máximo real en 0.10–0.15.

Dado que el perfil de error está dominado por falsos negativos (60 vs. 21 FP), **bajar el umbral es la dirección correcta**: aumentaría el recall a costa de precisión, que es el recurso abundante.

Corrección aplicada: el rango arranca en 0.05 y, si el óptimo vuelve a caer en cualquiera de los dos extremos, el código lo advierte en lugar de dejarlo pasar.

```
for thr in np.arange(0.05, 0.91, 0.05):
    ...
if OPT_THR <= SWEEP[0][0] + 1e-9:
    print("AVISO: el óptimo cayó en el borde inferior; ampliar el barrido.")
elif OPT_THR >= SWEEP[-1][0] - 1e-9:
    print("AVISO: el óptimo cayó en el borde superior; ampliar el barrido.")
```

El aviso importa más que el rango nuevo: ampliar el rango resuelve este caso, pero el chequeo de borde detecta el problema **la próxima vez**, sea cual sea el rango.

10.4 Otras limitaciones

- **Resolución de 320 px.** Causa más probable del bajo mAP@0.75 y de los FN en piletas pequeñas; la limitación de mayor impacto esperado.
- **Un solo modelo evaluado.** Sin comparación empírica con otras variantes (s, m) ni con otras configuraciones de transfer learning.
- **Sin validación cruzada.** Todas las métricas provienen de una única partición y semilla; no hay estimación de varianza.
- **Sesgo geográfico desconocido.** El dataset no documenta su cobertura espacial.

11. Mejoras futuras

Ordenadas por relación entre impacto esperado y esfuerzo.

Prioridad alta — ya corregido en el código, pendiente de reejecutar. Las tres correcciones de §10 están aplicadas pero **no reentrenadas**, porque requieren GPU. (1) **Reentrenar con el split 70/15/15**: duplica los datos de entrenamiento, el mayor impacto esperado. (2) **Rehacer el barrido desde 0.05**: costo nulo, y el análisis de errores indica que el óptimo está fuera del rango anterior. (3) **Recalcular la comparación `.pt` vs. `.onnx`** sobre el mismo conjunto. Las tres se resuelven con una sola corrida completa del pipeline corregido.

Prioridad media. (4) **Ejecutar la Etapa 4B** (experimento de capas congeladas, ya implementado): tres corridas `freeze=0`, `10` y `24` a igual número de épocas y semilla, comparando mAP, tiempo por época y memoria; convertiría §5 de exposición teórica en resultado medido. Costo estimado: 60–90 min en T4. (5) **Entrenar a `imgsz=640`**: vía directa para atacar el bajo mAP@0.75 y los FN en objetos pequeños. (6) **Aumentar el presupuesto de épocas**: el early stopping no se activó.

Prioridad baja. (7) TTA con *flip* horizontal (+1–2 puntos de mAP, al doble de costo). (8) Aumentación con *copy-paste*, apropiada para clases minoritarias. (9) Barrido de `lr0` y `weight_decay`. (10) Validación cruzada en 5 particiones. (11) Exportación a TFLite u OpenVINO para CPU o móviles.

12. Conclusiones

Sobre el resultado técnico. Un modelo de 2.51 M parámetros, entrenado 25 épocas en una GPU gratuita desde pesos de COCO, alcanza mAP@0.5 = 0.7846 y F1 = 0.8094 en detección de piletas sobre imágenes satelitales. Pesa 5.23 MB y corre a 104.8 FPS en GPU o ~27 en CPU. Para un presupuesto de aproximadamente una hora de cómputo, el resultado es sólido.

Sobre el carácter del modelo. La asimetría del error (60 FN contra 21 FP) lo define como conservador: confiable cuando avisa, pero omite una de cada cuatro piletas. La brecha entre mAP@0.5 (0.78) y mAP@0.75 (0.40) revela que el problema no es reconocer piletas sino delimitarlas con precisión, consecuencia esperable de entrenar a 320 px.

Sobre transfer learning. Se optó por fine-tuning completo por la distancia de dominio entre COCO y las imágenes aéreas. La decisión es defendible y los resultados la respaldan, pero **no fue validada experimentalmente**: sin una corrida comparativa con capas congeladas, la justificación es argumental y no empírica. Es la brecha metodológica más clara del trabajo. El experimento quedó implementado y probado (Etapa 4B), con la mecánica del parámetro `freeze` verificada sobre el modelo real —incluido el hallazgo de que el valor habitualmente citado, `freeze=22`, no corresponde a "solo la cabeza" en esta arquitectura—; falta ejecutarlo en GPU sobre el dataset completo.

Sobre el método. Los tres hallazgos de la auditoría comparten un patrón: **todos son fallas silenciosas**. El split incorrecto, el delta de ONNX no comparable y el barrido truncado producen números plausibles, atravesaron la ejecución completa sin generar un solo error y quedaron registrados en los archivos de resultados. Por eso las correcciones de §10 no se limitan a arreglar cada caso: incorporan *asserts* de proporción, un aviso de óptimo en borde y el registro explícito del conjunto usado en cada comparación. La idea es que el próximo error del mismo tipo **haga ruido** en vez de llegar hasta el informe.

Esto conecta con el incidente del mapeo de clases de §2.3, donde un pipeline sin excepciones entrenó un modelo sobre cero instancias de la clase objetivo. La conclusión transversal es que **en visión por computadora el riesgo principal no es el código que falla, sino el que funciona sobre premisas equivocadas**. Las verificaciones que resultaron decisivas fueron las más elementales: graficar una muestra de datos, controlar que los conteos cierren, confirmar que un valor óptimo no caiga en el borde del rango buscado.

Un detector con mAP de 0.78 es un resultado. Saber por qué es 0.78, qué parte del pipeline lo limita y cuál es la próxima corrección que más lo movería es el resultado que importa.

Anexo — Procedencia de los datos. Las métricas provienen de `metricas.csv` (16 métricas de detección, decisión y costo), `trade_off_onnx.csv` (comparación `.pt` vs. `.onnx`) y `results.csv` (pérdidas y mAP por época, incluye `train/dfl_loss`). Las figuras son `curvas_entrenamiento.png`, `f1_vs_umbral.png`, `curvas_pr.png`, `comparativa_gt_vs_pred.png`, `eda_ejemplos.png` y `galeria_10k.png`. Los valores de TP, FP y FN de §8.1 se reconstruyeron algebraicamente a partir de la precisión y el recall registrados, verificando consistencia con la única solución entera del sistema. Todas las métricas provienen de ejecución real del pipeline: ningún número fue estimado ni transcrito a mano.

Los recuentos de parámetros entrenables por valor de `freeze` (§5.3) **no provienen de esa corrida** sino de cargar `yolov5nu.pt` y ejecutar el *trainer* de Ultralytics con cada configuración, midiendo `requires_grad` en el callback `on_train_start`. Se verifican también contra el código fuente del framework. Las métricas de mAP de las estrategias con capas congeladas **no se reportan porque no se midieron** (§5.6).